



1 Why Playwright Changes Everything

- ✗ Flaky selectors

✗ Slow execution

✗ Single browser only

✗ Complex waits

➤

✓ Auto-waiting built-in

✓ Parallel execution

✓ Chromium, Firefox, WebKit

✓ Web-first assertions

From Selenium to Modern Automation

Playwright is a next-gen end-to-end testing framework by Microsoft. It offers auto-waiting, network interception, multi-browser support, and codegen—all with TypeScript-first design. It's the difference between writing fragile scripts and building reliable automation.

Links: [🔗 playwright.dev](#) [🔗 GitHub Repo](#)

2 Core Features & Capabilities

Multi-Browser Testing
Chromium, Firefox, WebKit. Test on all engines with one API. Mobile emulation included.
Impact: "One test suite, all browsers"

Auto-Waiting & Assertions
No manual waits. Playwright auto-waits for elements to be actionable before performing actions.
Impact: "Zero flaky tests from timing"

Network Interception
Mock APIs, modify requests/responses, test offline mode, HAR recording & replay.
Impact: "Full control over network layer"

Built-in Test Runner
Parallel execution, retries, reporters, fixtures, projects—all out of the box.
Impact: "No extra frameworks needed"

Links: [🔗 API Reference](#) [🔗 Best Practices Guide](#)

3 Test Structure & Workflow

Setup

- Configure browsers
- Set base URL
- Define projects
- Setup fixtures

➤

Locate

- getByRole()
- getByText()
- getByTestId()
- CSS/XPath

➤

Act

- click(), fill()
- type(), press()
- hover(), check()
- selectOption()

➤

Assert

- toBeVisible()
- toHaveText()
- toHaveURL()
- toHaveCount()

```
// Basic Test Structure import { test, expect } from '@playwright/test'; test('user can login', async ({ page }) => { await page.goto('https://example.com/login'); await page.getByLabel('email').fill('user@test.com'); await page.getByLabel('password').fill('secret'); await page.getByRole('button', { name: 'Sign In' }).click(); await expect(page).toHaveURL(/dashboard/); });
```

Links: [🔗 Writing Tests](#) [🔗 Test Generator](#)

4 Getting Started

Recommended Setup:

- Install Node.js (v18+)
- `npm init playwright@latest`
- Choose TypeScript, add GitHub Actions
- Install browsers: `npx playwright install`
- Run tests: `npx playwright test`

Requirements:

- Node.js 18+ / npm or yarn
- VS Code + Playwright Extension
- TypeScript 4.x+

Project Structure:

```
|— playwright.config.ts |— package.json |— tests/ |— login.spec.ts |— home.spec.ts |— pages/ // Page Objects |— LoginPage.ts |— HomePage.ts |— fixtures/ |— base.ts |— utils/ |— helpers.ts |— test-results/
```

Links: [🔗 Installation Guide](#) [🔗 VS Code Extension](#)

5 Page Object Model (POM)

```
// pages/LoginPage.ts import { Page, Locator } from '@playwright/test'; export class LoginPage { readonly emailInput: Locator; readonly passwordInput: Locator; readonly submitBtn: Locator; constructor(public page: Page) { this.emailInput = page.getByLabel('email'); this.passwordInput = page.getByLabel('password'); this.submitBtn = page.getByRole('button', {name: 'Login'}); } async login(email: string, password: string) { await this.emailInput.fill(email); await this.passwordInput.fill(password); await this.submitBtn.click(); } }
```

Best Practice: Use POM for reusable, maintainable test code. [🔗 POM Guide](#)

6 Locator Strategies (Recommended Order)

User-Facing Locators (Preferred ✓)

Method	Example	Use Case
getByRole()	page.getByRole('button', { name: 'Submit' })	Buttons, links, headings
getByText()	page.getByText('Welcome back')	Visible text content
getByLabel()	page.getByLabel('Email')	Form inputs with labels
getByPlaceholder()	page.getByPlaceholder('Search...')	Input placeholders
getByAltText()	page.getByAltText('Logo')	Images with alt text
getByTitle()	page.getByTitle('Settings')	Title attributes
getByTestId()	page.getByTestId('nav-menu')	data-testid attributes

CSS & XPath (Fallback)

Method	Example
locator()	page.locator('.btn-primary') · page.locator('#email')
locator() XPath	page.locator('//*[@@class="card"]')
nth()	page.locator('.item').nth(2)
filter()	page.locator('.row').filter({ hasText: 'Active' })

Links: [🔗 Locators Guide](#) [🔗 LocatorLabs Chrome Extension](#)

7 Essential Actions & Assertions

Actions (page / locator)		Assertions (expect)	
Action	Method	Assertion	Method
Navigate	page.goto(url)	Visible	toBeVisible()
Click	locator.click()	Hidden	toBeHidden()
Double Click	locator.dblclick()	Text	toHaveText('hello')
Right Click	locator.click({button: 'right'})	Contain Text	toContainText('he')
Fill Input	locator.fill('text')	Value	toHaveValue('abc')
Clear + Type	locator.pressSequentially('hi')	URL	toHaveURL(/dash/)
Check/Uncheck	locator.check()	Title	toHaveTitle('Home')
Select Option	locator.selectOption('val')	Enabled	toBeEnabled()
Hover	locator.hover()	Checked	toBeChecked()
Upload File	locator.setInputFiles('f.pdf')	Count	toHaveCount(5)
Drag & Drop	locator.dragTo(target)	Attribute	toHaveAttribute('href', '/')
Screenshot	page.screenshot({path: 's.png'})	CSS	toHaveCSS('color', 'red')

8 Advanced Features

API Testing

```
const response = await request.get('/api/users'); expect(response.status()).toBe(200); const body = await response.json(); expect(body.users).toHaveLength(10);
```

Network Mocking

```
await page.route('*/api/users', route => route.fulfill({ body: JSON.stringify({name:'Mock'})}), {contentType: 'application/json'});
```

Video Recording

Screenshots

Trace Viewer

Auto Retries

Fixtures

Mobile Emulation

Geolocation

Auth State

Downloads

Multi-tab

iFrames

Dialogs

🌟 Playwright MCP (AI + Browser Automation)

@playwright/mcp — Official MCP server by Microsoft. Enables LLMs to interact with web pages via structured accessibility snapshots needed!

Quick Setup (Claude / Cursor / VS Code)

```
// Add to MCP config (claude, cursor, etc.) { "mcpServers": { "playwright": { "command": "npx", "args": ["@playwright/mcp@latest"] } } }
```

What PW MCP Can Do

- 🌐 **Navigate** — browse pages, click, fill forms
- 📸 **Screenshot** — capture page state
- 🔍 **Snapshot** — read accessibility tree
- ⚡ **Execute JS** — run scripts in browser
- ✅ **Generate Tests** — AI writes PW tests
- 🛠️ **Works with** — Claude, Cursor, Copilot, Windsurf

Links: [🔗 GitHub: playwright-mcp](#) [🔗 MCP Videos](#)

9 Configuration & CLI Commands

CLI Commands

Command	Description
<code>npx playwright test</code>	Run all tests
<code>npx playwright test login</code>	Run specific file
<code>--headed</code>	Run in browser UI
<code>--project=chromium</code>	Specific browser
<code>--grep "login"</code>	Filter by title
<code>--workers=4</code>	Parallel workers
<code>npx playwright codegen</code>	Generate test code
<code>npx playwright show-report</code>	Open HTML report
<code>npx playwright show-trace</code>	Open trace viewer

playwright.config.ts (Key Options)

```
export default defineConfig({ testDir: './tests', timeout: 30_000, retries: 2, workers: 4, reporter: [ ['html'], ['allure-playwright'] ], use: { baseURL: 'https://example.com', headless: true, screenshot: 'on-only-on-failure', video: 'retain-on-failure', trace: 'on-first-retry', }, projects: [ { name: 'chromium', use: { ...devices['Desktop Chrome'] }, { name: 'firefox', use: { ...devices['Desktop Firefox'] }, { name: 'mobile', use: { ...devices['iPhone 14'] }, }, ], });
```

10 Hooks, Fixtures & Annotations

Hook/Annotation	Usage
<code>test.beforeAll()</code>	Run once before all tests
<code>test.beforeEach()</code>	Run before each test
<code>test.afterEach()</code>	Run after each test
<code>test.afterAll()</code>	Run once after all tests
<code>test.describe()</code>	Group related tests
<code>test.skip()</code>	Skip a test
<code>test.only()</code>	Run only this test
<code>test.slow()</code>	3x default timeout

Custom Fixtures

```
import { test as base } from '@playwright/test'; export const test = base.extend({ loginPage: LoginPage })({ loginPage: async ({ page }, use) => { await use(new LoginPage(page)); }, });
```

Auth Storage State

```
// Save auth state await page.context().storageState({ path: 'auth.json' }); // Reuse in config: storageState: 'auth.json'
```

Official Resources:

[🔗 playwright.dev/docs/intro](#) — Full Documentation

[🔗 playwright.dev/docs/api](#) — API Reference

[🔗 playwright.dev/docs/codegen](#) — Test Generator

Key Reporters:

Reporter	Command / Config	Best For
HTML	reporter: 'html'	Local dev
Allure	allure-playwright	CI/CD dashboards
JUnit	reporter: 'junit'	Jenkins / CI
JSON	reporter: 'json'	Custom processing

Quick Reference:

Keyboard Shortcut	Action
Ctrl+Shift+P	VS Code: Run Tests
---debug	Debug mode with Inspector
---ui	Open Playwright UI mode
page.pause()	Pause test execution

